Building Your Next App with .NET MAUI

David Ortinau | Principal Program Manager | Microsoft
@davidortinau | david.ortinau@microsoft.com



.NET MAUI is the most productive way to develop native apps that perform great on any device that runs Android, iOS, macOS, or Windows from a single codebase.

Productive

- .NET Hot Reload
  - XAML
  - C# method body replacement
  - Razor and CSS
  - With debugger and without
- Windows + iOS Device
- Single Project
- Handlers are easy to override or extend
- Maui.Graphics (Experimental)
- VS – Live Visual Tree, Profiler, Debugger, etc.

Perform(ance)

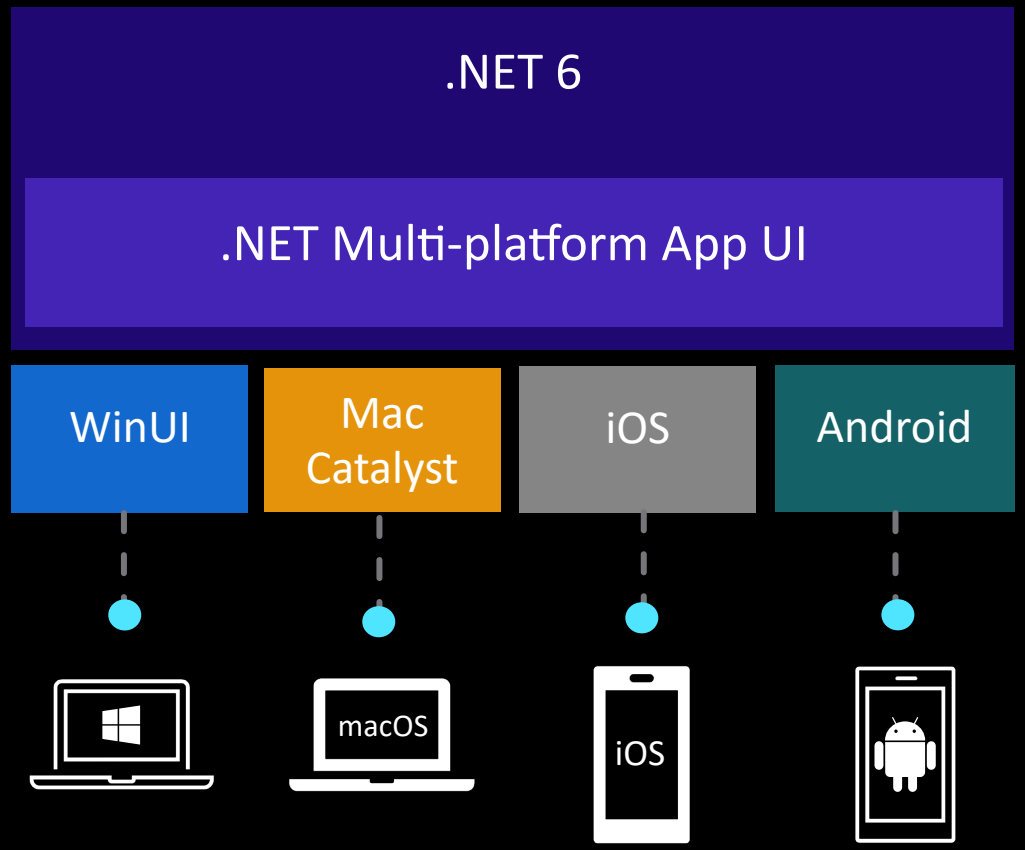
- Handlers are lightweight and don’t use INPC
- Layout Managers reduce measure loops
- Application startup uses explicit registration, no reflection
- Source generators
- .NET 6 BCL perf wins
- Maui.Graphics (Experimental)

Single Codebase

- Single Project – all .NET
- Multi-targeting
- Images, Fonts, Raw assets, etc. all shared
- Single application point of entry
- Simplified binding of native libs for app devs

.NET MAUI is the evolution of Xamarin.Forms extended from mobile to desktop scenarios with UI controls rebuilt from the ground up for performance and extensibility.

.NET Multi-platform App UI



Cross-platform, native UI

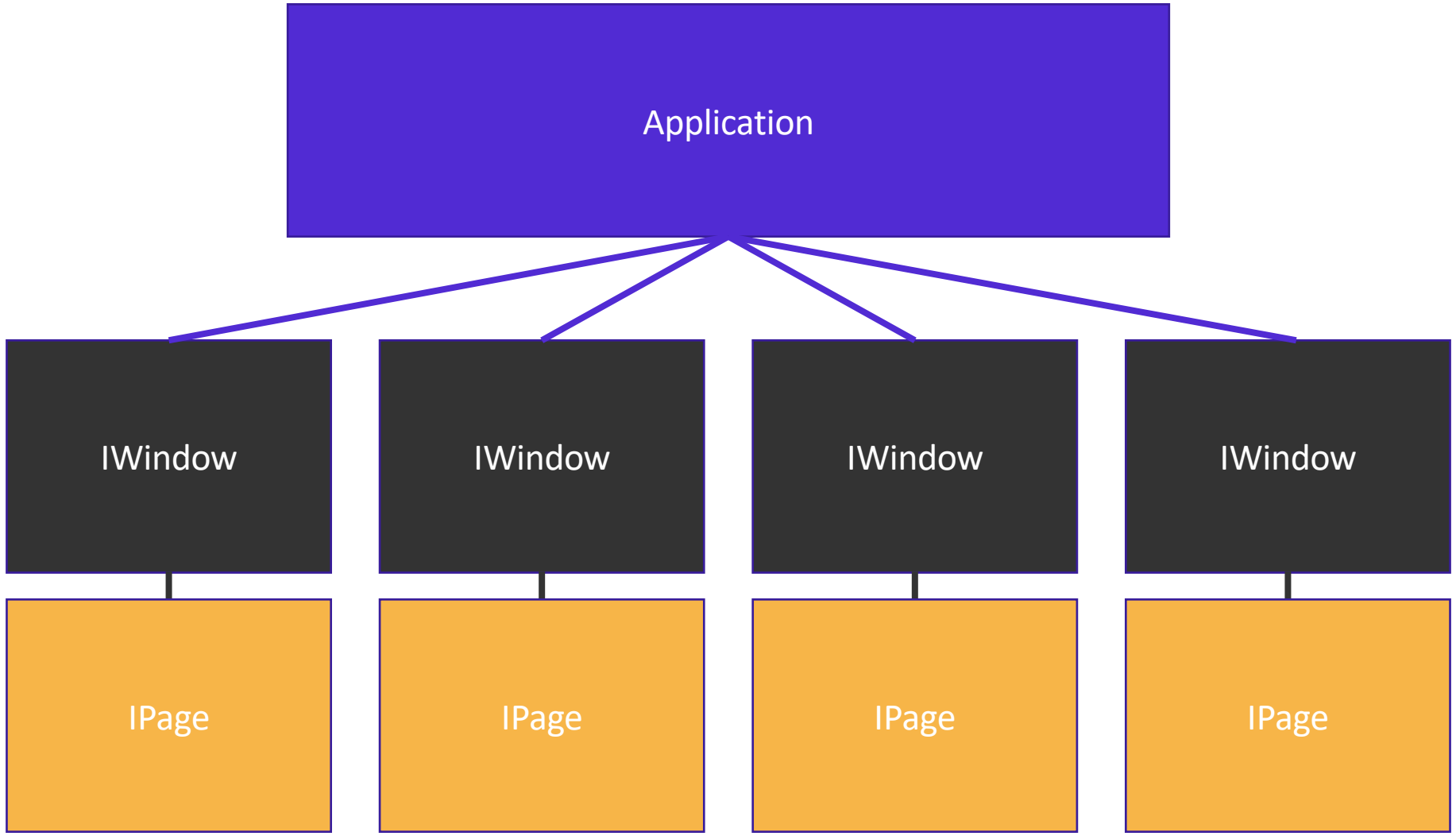
Single project system, single codebase

Deploy to multiple devices, mobile & desktop

General availability in .NET 6

github.com/dotnet/maui

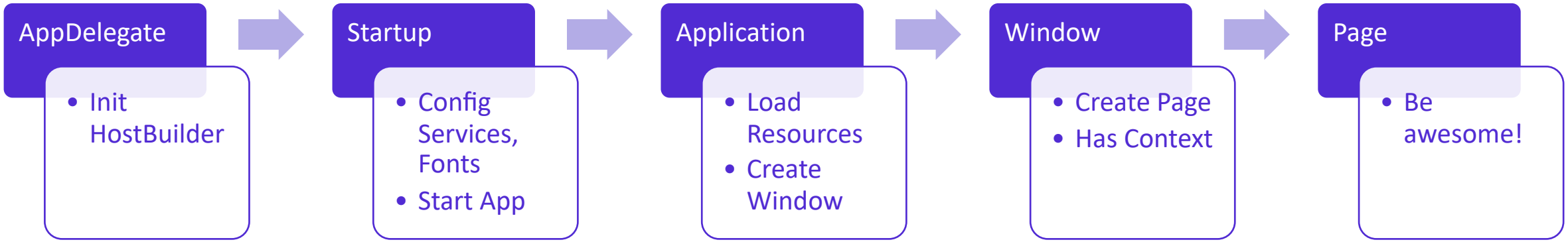
Let’s Code!


Application
IWindow
IWindow
IWindow
IWindow
IPage
IPage
IPage
IPage

Application

- Can use App.xaml
- Or provide your own IApplication (e.g. Comet)
- Startup.cs inits application
  - No more reflecting over every assembly
  - RegisterCombatibiltyRenderers() we will reflect over 1 assembly to wire up
  - Use RegisterCompatibilityRenderer<Type,Type> for fine control

Ready, Set, Go!


AppDelegate
• Init HostBuilder
Startup
• Config Services, Fonts
• Start App
Application
• Load Resources
• Create Window
Window
• Create Page
• Has Context
Page
• Be awesome!

Lifecycle

Cross Platform Lifecycle

The Application, Window, Page and View classes contains virtual methods that can be overridden to respond to lifecycle changes.

```
protected override void OnCreated()
{
    Debug.WriteLine ("OnCreated");
}
protected override void OnResumed()
{
    Debug.WriteLine ("OnResumed");
}
protected override void OnPaused()
{
    Debug.WriteLine ("OnPaused");
}
```

Native Platform Lifecycle

.NET MAUI also will allow to hook into native life cycle events without the complexities of wiring up a renderer so if require finer platform level hooks they can hook in in an easy way.

```
appBuilder.ConfigureAndroidLifecyle(lifecycle => {
              lifecycle
                            .OnCreate((ctx, state) => { AppCenter.Init(ctx, state);  })
                            .OnStart(() => { AppCenter.StartTracking(); });
});
```

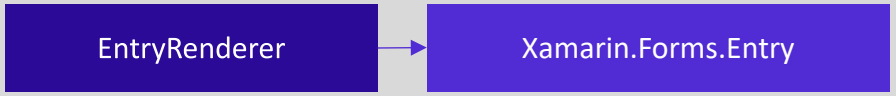
https://github.com/dotnet/maui/issues/30

Evolving the Architecture

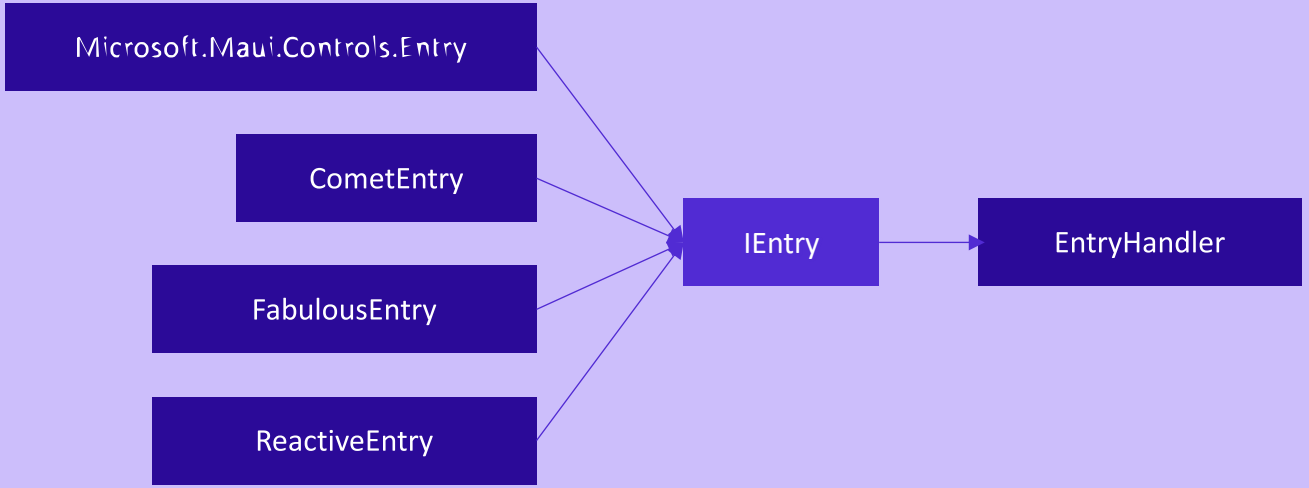
Renderers are tightly coupled to Xamarin.Forms and Xamarin.Forms components

Handlers are unwrapped, decoupled, portable, and reusable

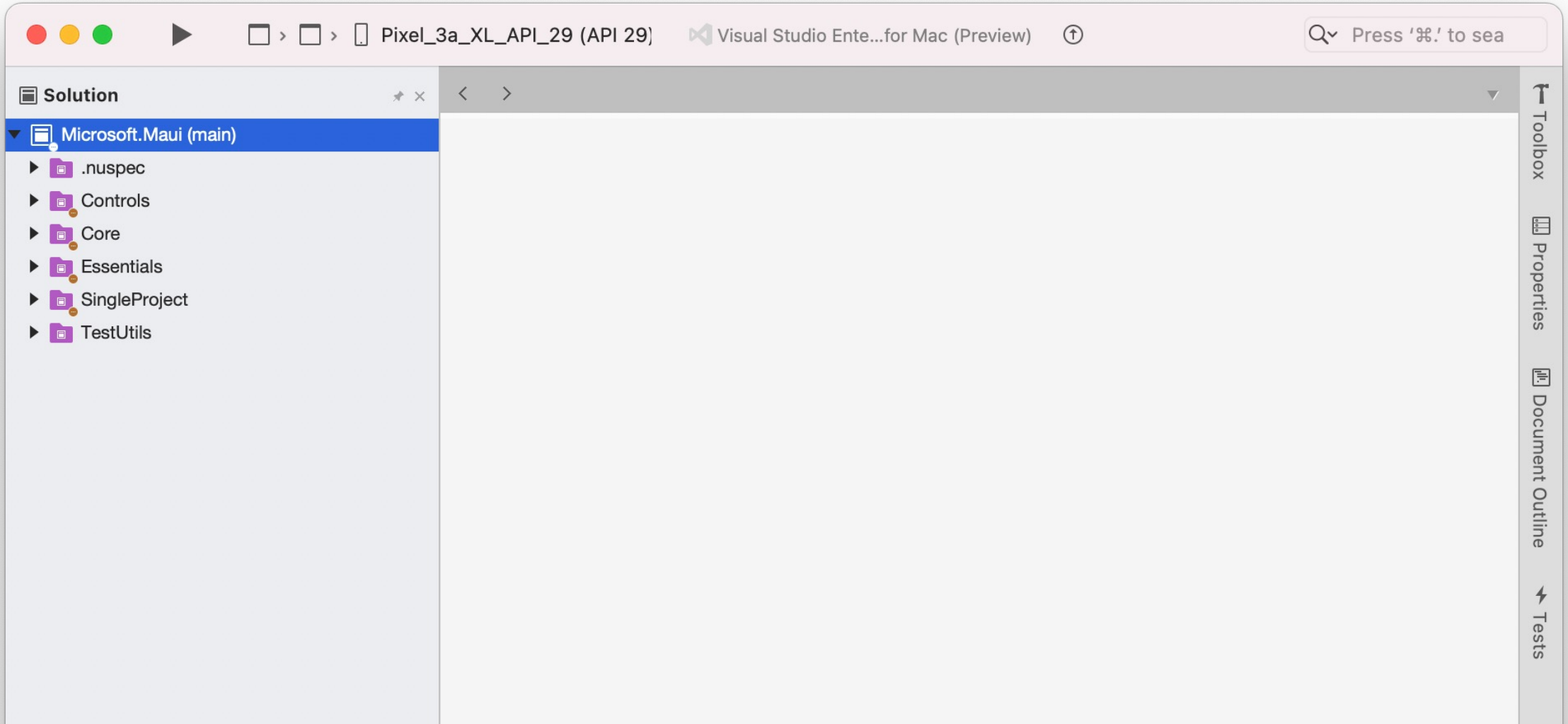
OLD



NEW



Entry : IEntry



Extending Controls

"So…if I'm not creating Custom Renderers, Effects, and PlatformSpecifics anymore…

how do I _____?"

Goodbye, Android Entry Underline

```
#if ANDROID
Handlers.EntryHandler
    .EntryMapper[nameof(IEntry.BackgroundColor)] = (h, v) =>
    {
        (h.NativeView as global::Android.Views.Entry).UnderlineVisible = false
    };
#endif
```

#ifdef if platform specific

Access the handler from anywhere

Dictionary of all props

Debug Rainbows

```
Handlers.ViewHandler
    .ViewMapper[nameof(IView.BackgroundColor)] = (h, v) ⇒
    {
      (h.NativeView as global::Android.Views.View).BackgroundColor
              = RandomColor();
    };
```

Make your a11y testing life easier!

```
#if ENABLE_TEST_CLOUD
Handlers.ViewHandler
    .ViewMapper[nameof(IView.AutomationId)] = (h, v) =>
    {
        (h.NativeView as global::Android.Views.View).ContentDescription
            = v.AutomationId;
    };
#endif
```

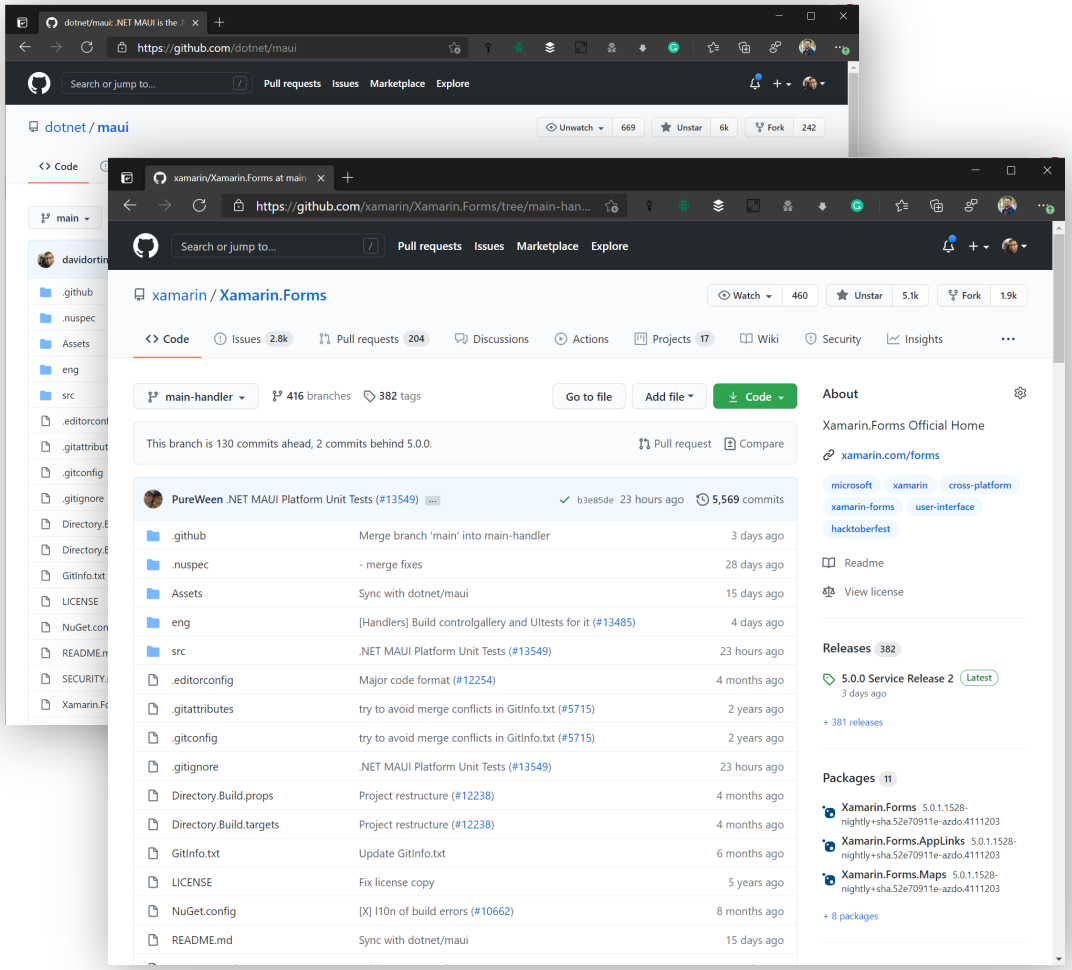
Layout Optimizations

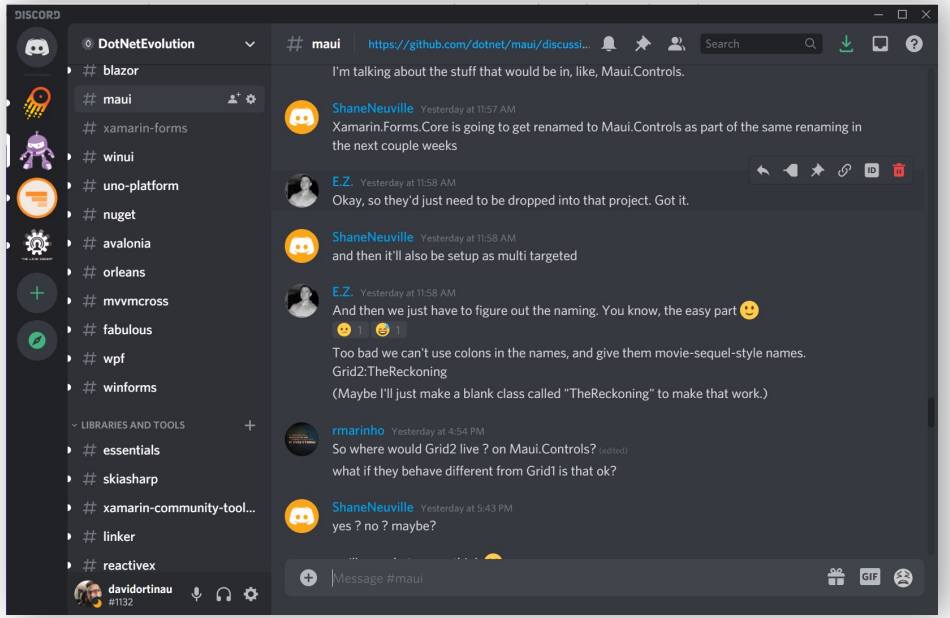
- Reducing layout passes
- Rewriting GridLayout to be .NET MAUI 1st
- VerticalStackLayout and HorizontalStackLayout available as optimized versions
- Remove/minimize dependence on BindableObject
- WidthRequest -> Width
- HeightRequest -> Height
- Shift to doing the work in LayoutManager class with easy to override methods

Coming EVERYWHERE in Preview 5→6

- Brushes
- Shapes
- Clipping
- Borders
- Corners
- Shadows

Where is this happening?

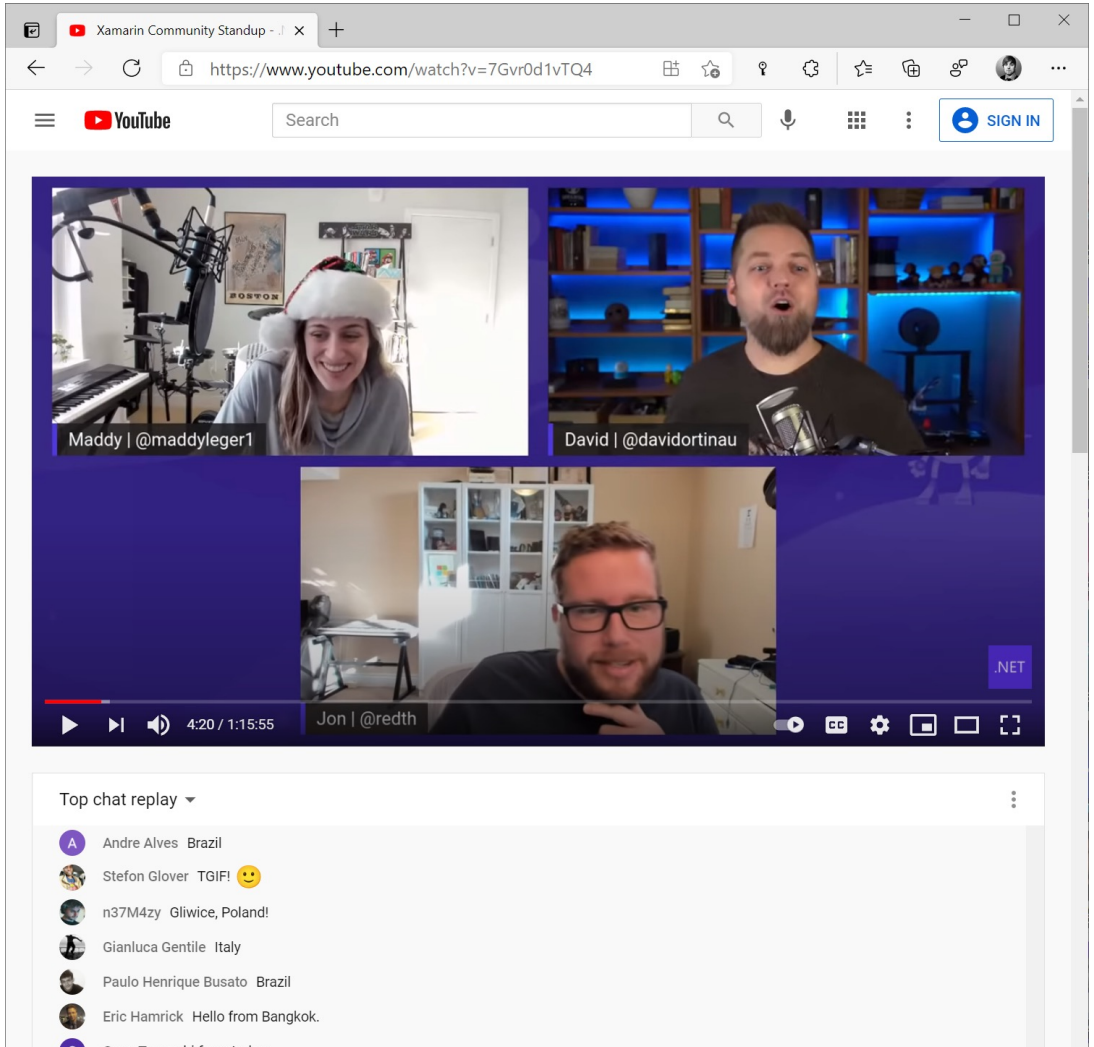




https://aka.ms/discord-maui

https://github.com/dotnet/maui

.NET Community Standup



https://live.dot.net

1st Thursday of the Month @ Noon PT / 3 ET

- Community News
- Contributions
- .NET MAUI Status Report
- Shenanigans!

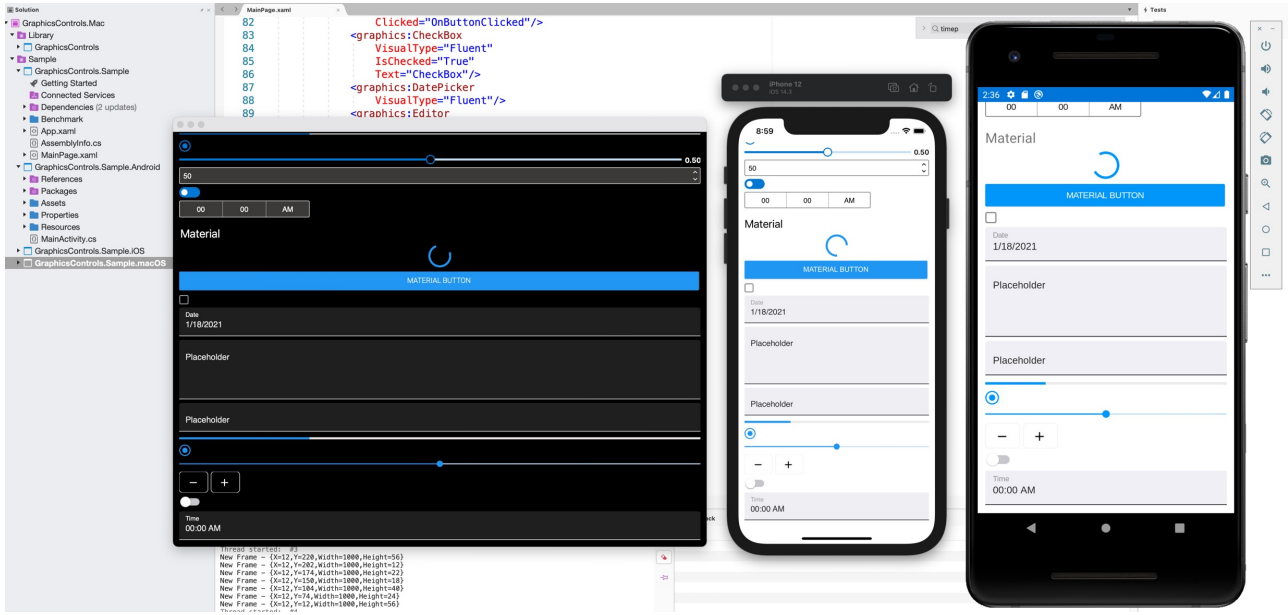
OSS Library Maintainer - Monthly Sync

- 1st Tuesday of the Month @ Noon PT / 3 ET
- Teams Call to deep dive on architecture and ask ?s
- Current representation from:
  - FreshMvvm
  - Mobile Build Tools
  - MvvmCross
  - Prism
  - ReactiveUI
  - Shiny
  - Xamarin Community Toolkit
  - Uno Platform

To join:
david.ortinau@microsoft.com

Experiment Graphics

Microsoft.Maui.Graphics is a cross-platform graphics library for iOS, Android, Windows, macOS, Tizen and Linux.



In an **experiment**, we are evaluating the possibility of use Microsoft.Maui.Graphics to create **drawn controls** and use it for Visual allowing to choose between Cupertino, Fluent and Material.

https://github.com/dotnet/Microsoft.Maui.Graphics

https://github.com/dotnet/GraphicsControls

Experiment Mobile Blazor Bindings

- Updating for .NET 6
- Adopting .NET MAUI
- BlazorWebView will go GA for Blazor Hybrid and .NET MAUI in .NET 6
- Future work to expand components will be evaluated later

Comet

- Updating for .NET 6
- Adopting .NET MAUI handlers
- Hot Reload w/ VS Code
- Source generators
- Updating to allow for manual updates vs observables
  - Aka magic or no magic

Thanks for joining!

David Ortinau | Principal Program Manager | Microsoft

david.ortinau@microsoft.com | @davidortinau

